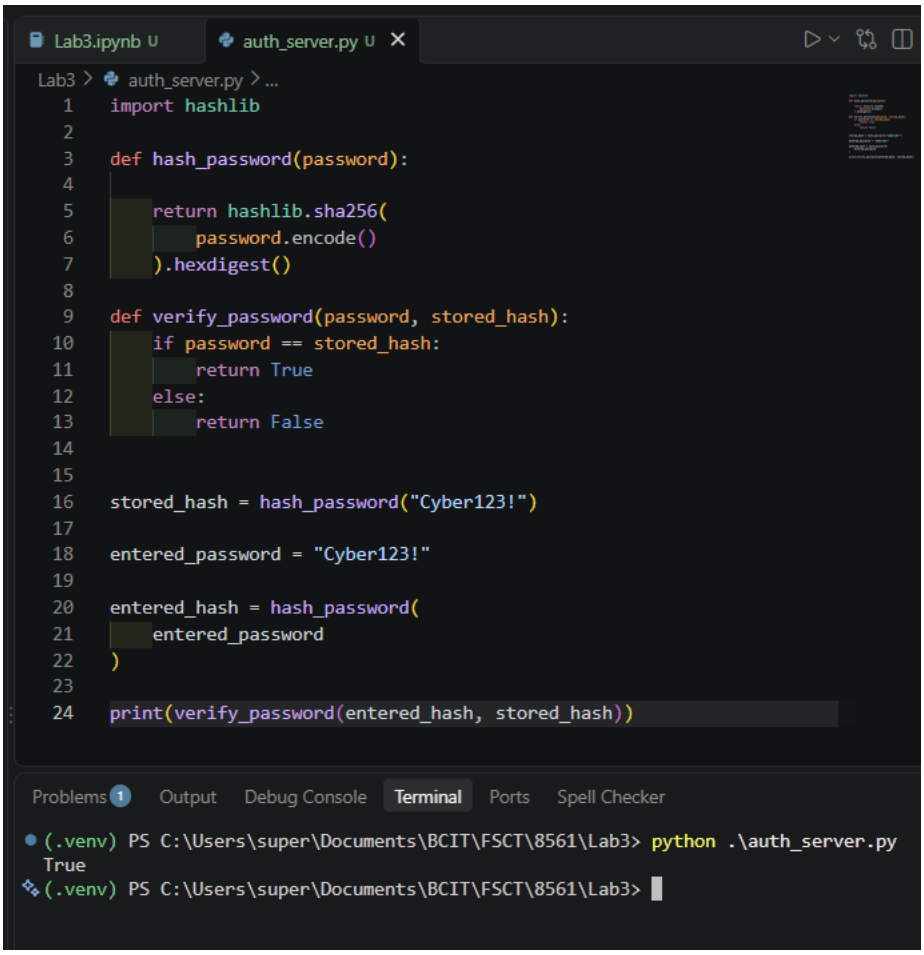
Braeden Venne

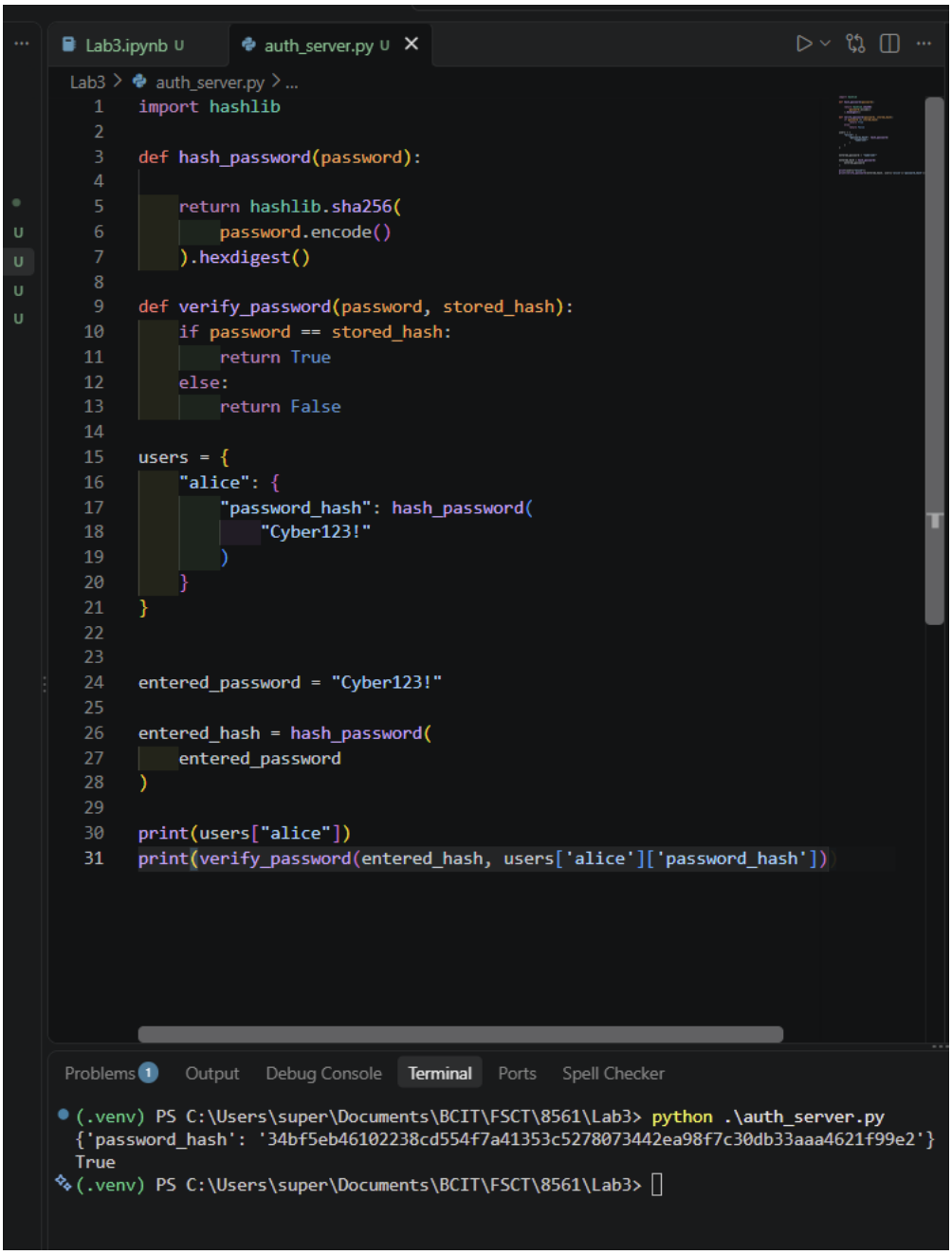
Lab 3

Part 2 – Turn Hashing into a Function



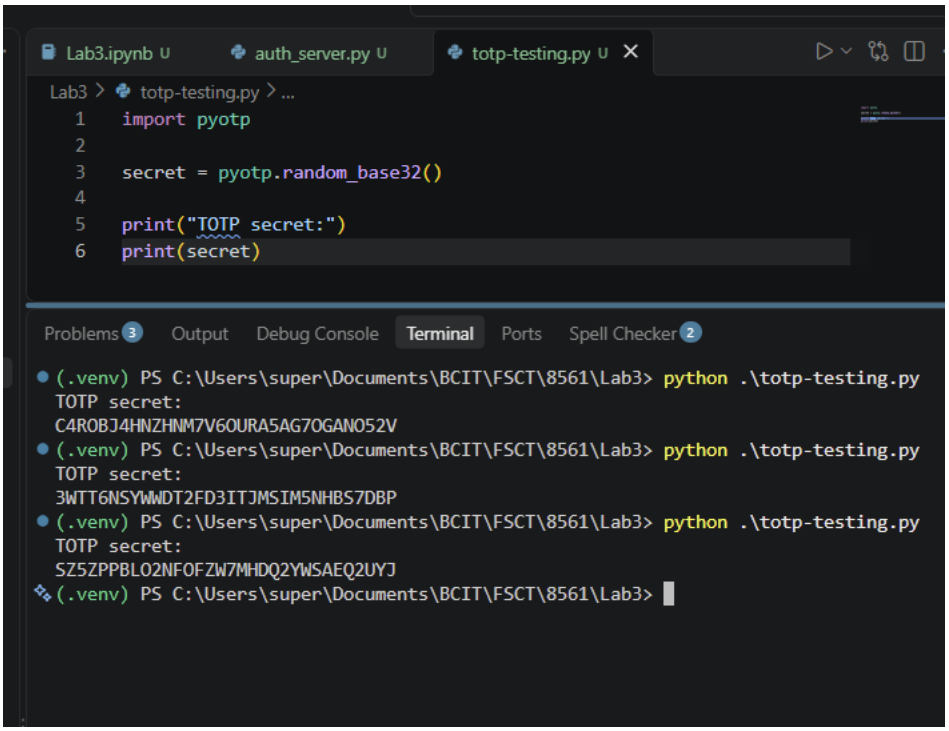
Braeden Venne

Part 3 – Create a Simple User Database



Braeden Venne

Part 4 / 5 – Introduce and Generate TOTP With pyotp

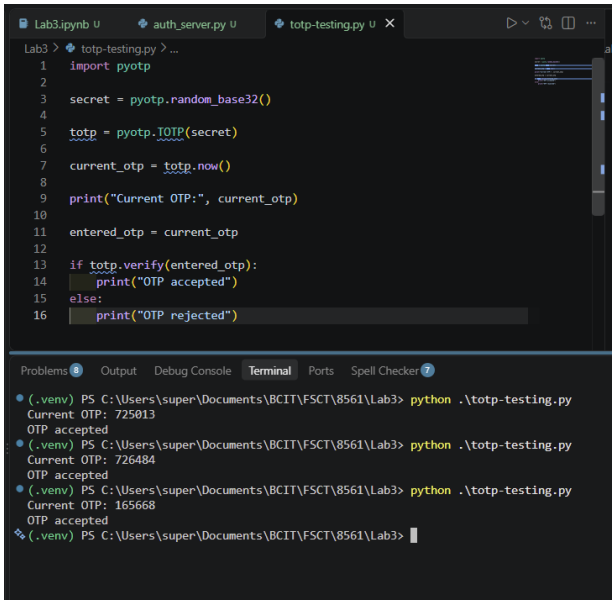


- Looking at the screenshot, pyotp.random_base32() generates a new secret every time

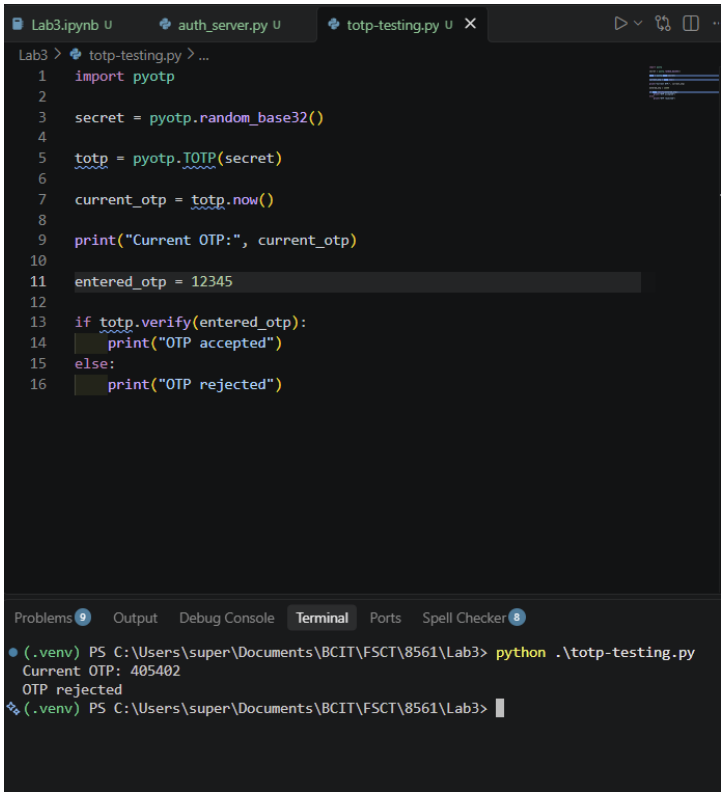
Braeden Venne

Part 6 – Generate and Verify a TOTP

1. Current OTP

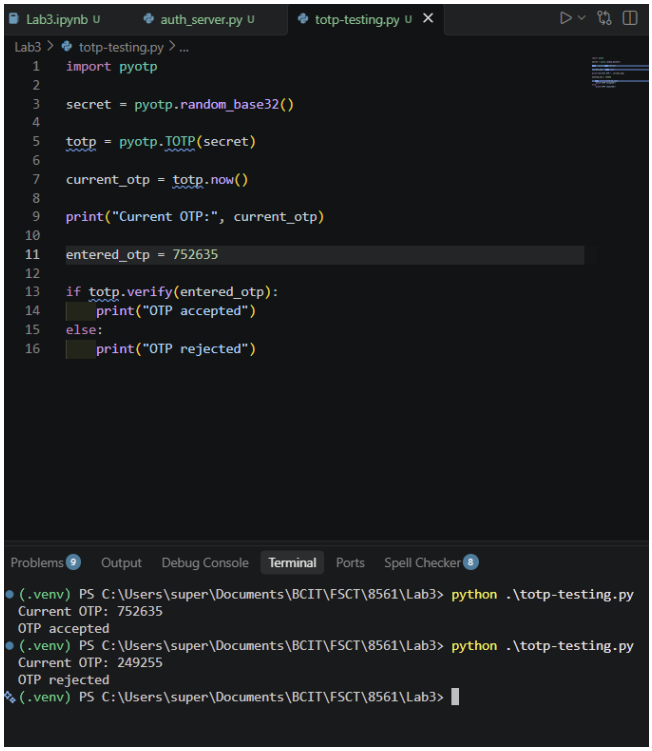


2. Incorrect OTP

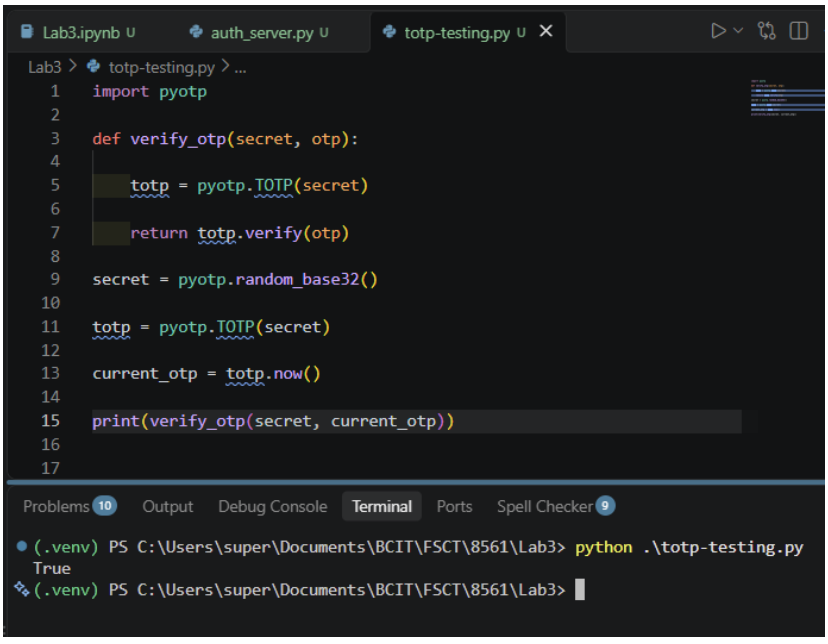


Braeden Venne

3. Previous OTP

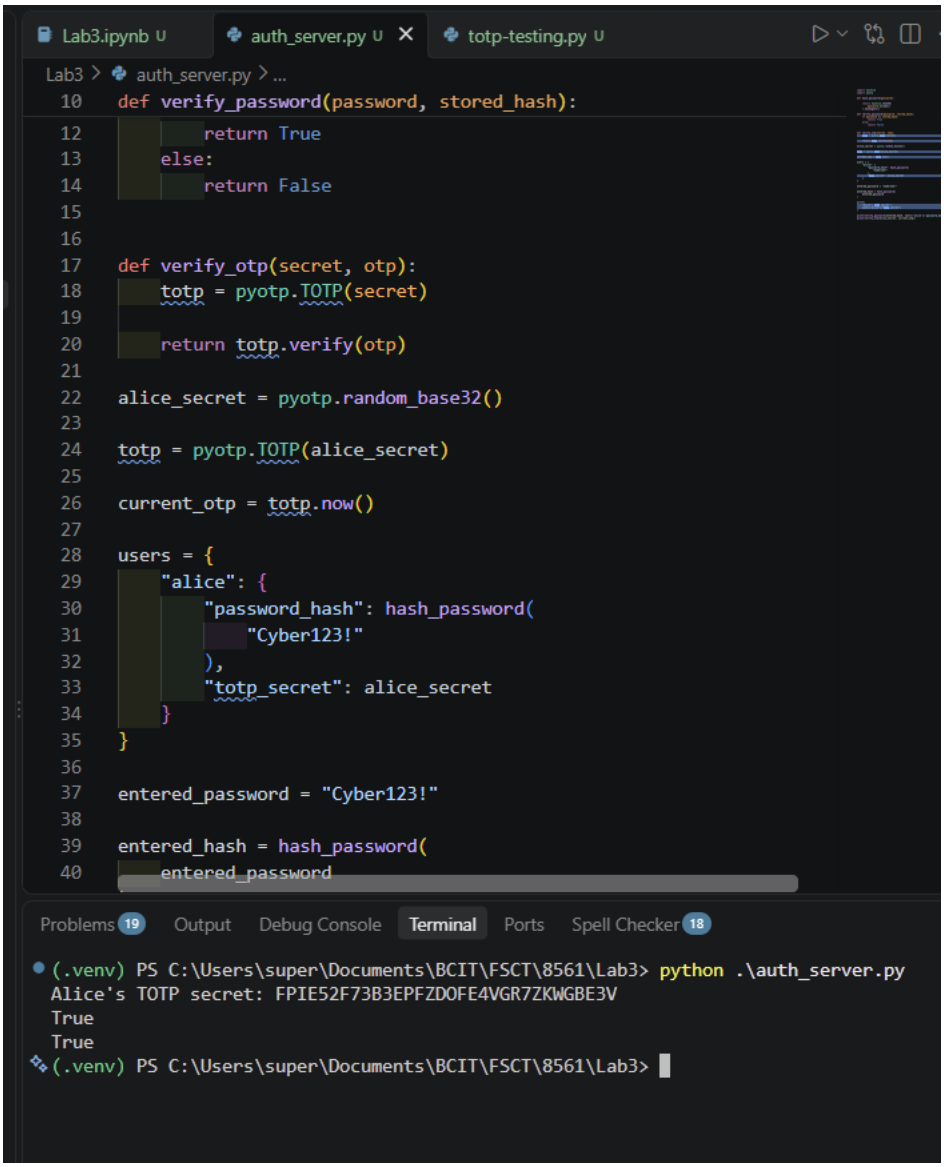


Created function



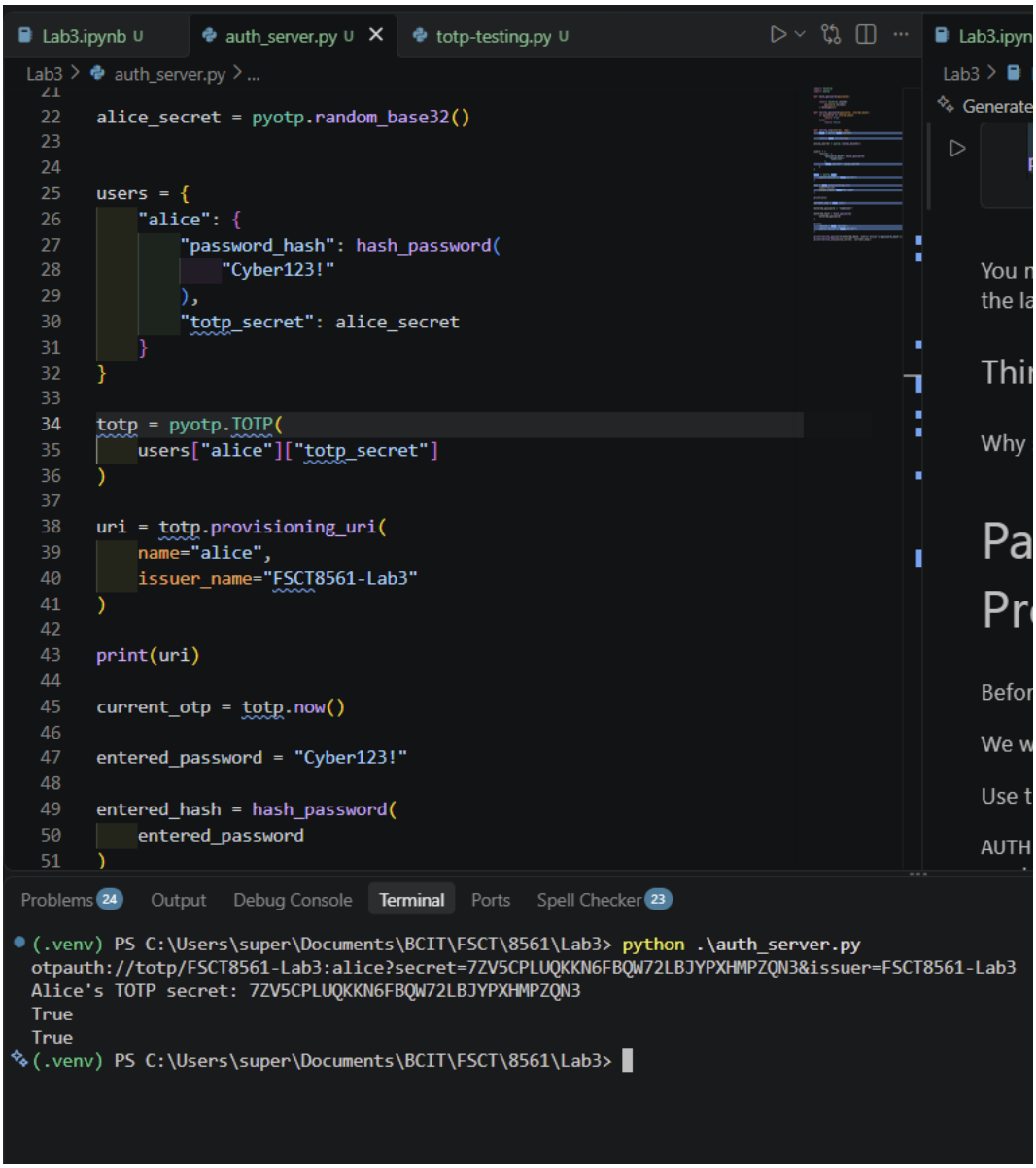
Braeden Venne

Part 7 – Add the TOTP Secret to the User Account

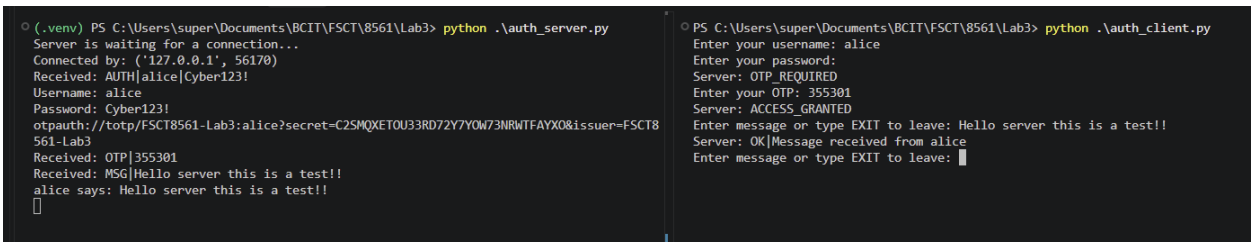


Braeden Venne

Part 8 – Optional Provisioning URI



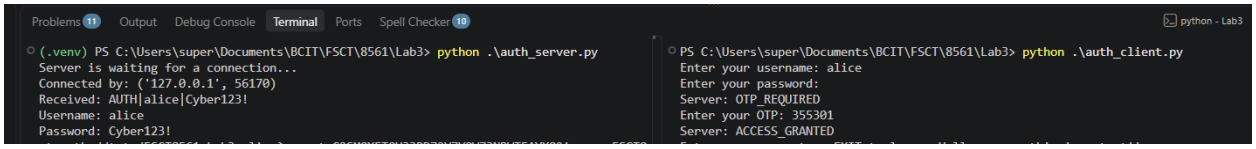
Part 9, 10, 11, 12 – Completing the System



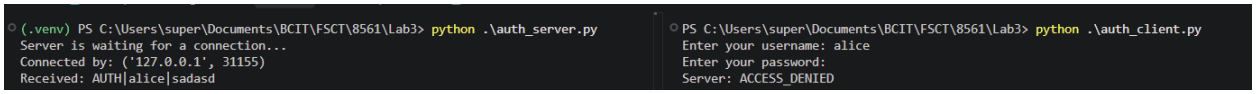
Braeden Venne

Part 13 – Security and Robustness Tests

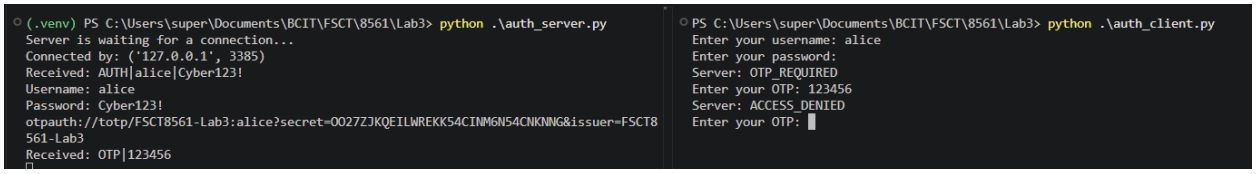
- Test 1 – Correct Password + Correct OTP



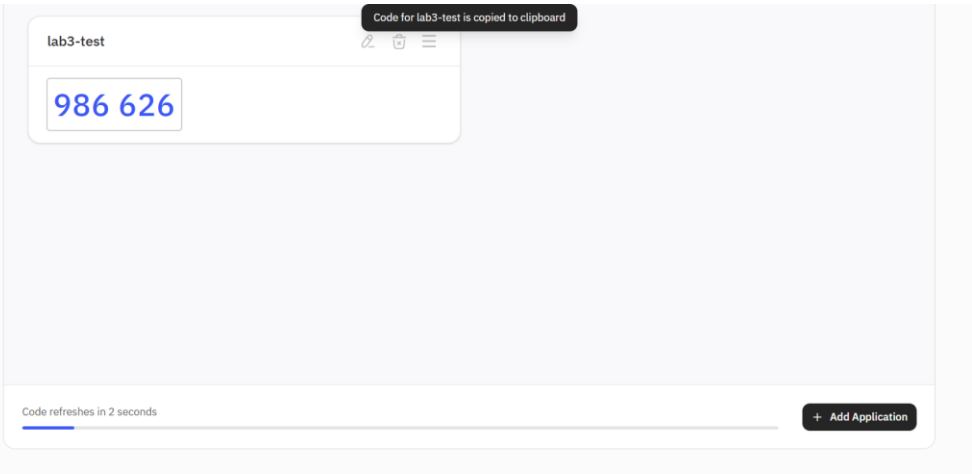
- Test 2 – Incorrect Password



- Test 3 – Incorrect OTP



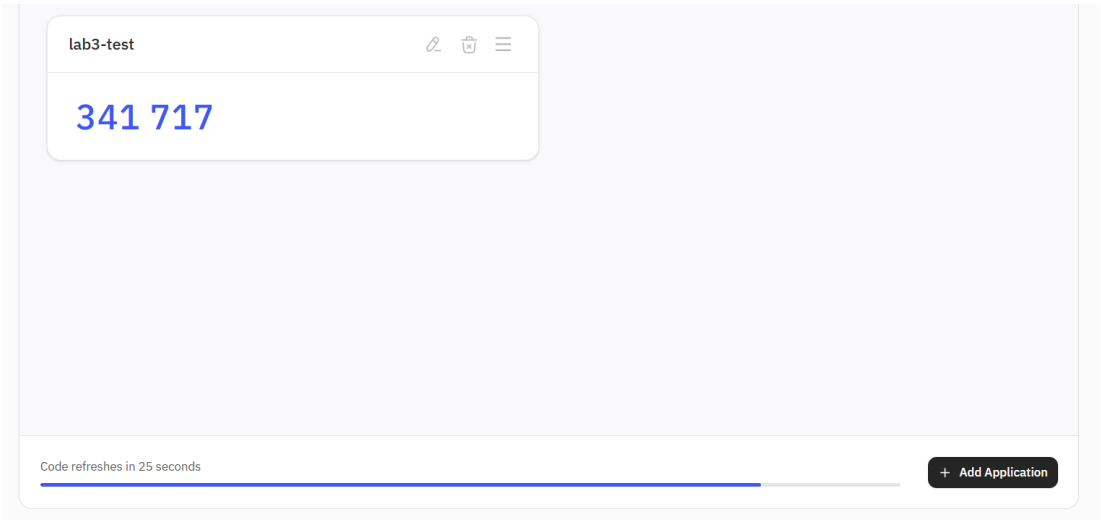
- Test 4 – Expired OTP
  - First generated an OTP then let it expire and attempted entering the OTP into the terminal

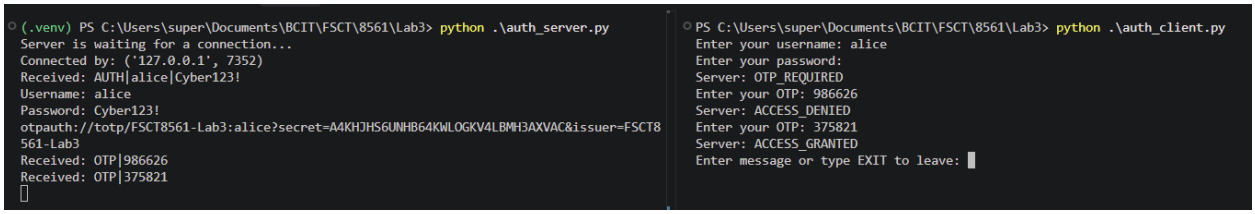




  - Screenshot above shows expected “ACCESS_DENIED” error
  - Tried with a newly generated OTP that was NOT expired:

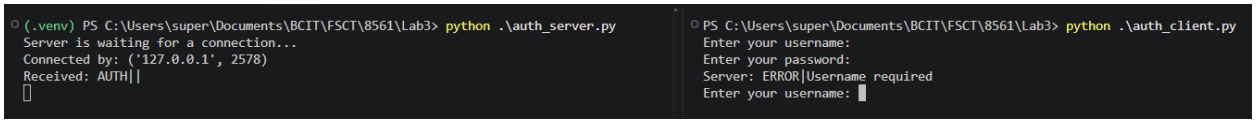
Braeden Venne



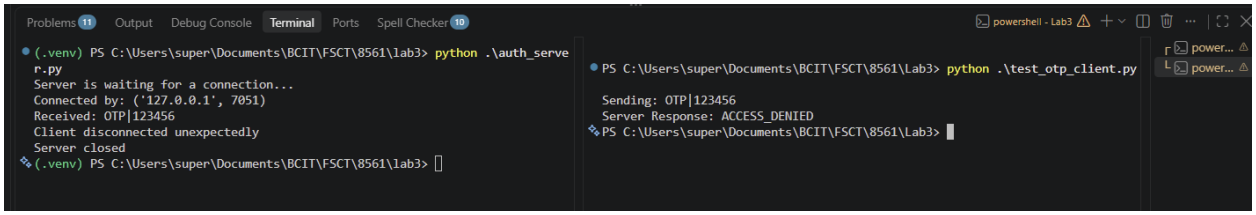


- Screenshot above shows that it is working as intended

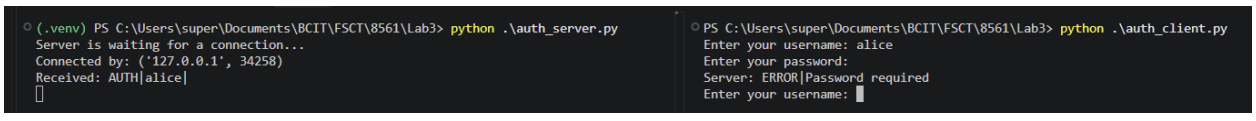
- Test 5 – Invalid Username



- Test 6 – OTP Before Password Verification

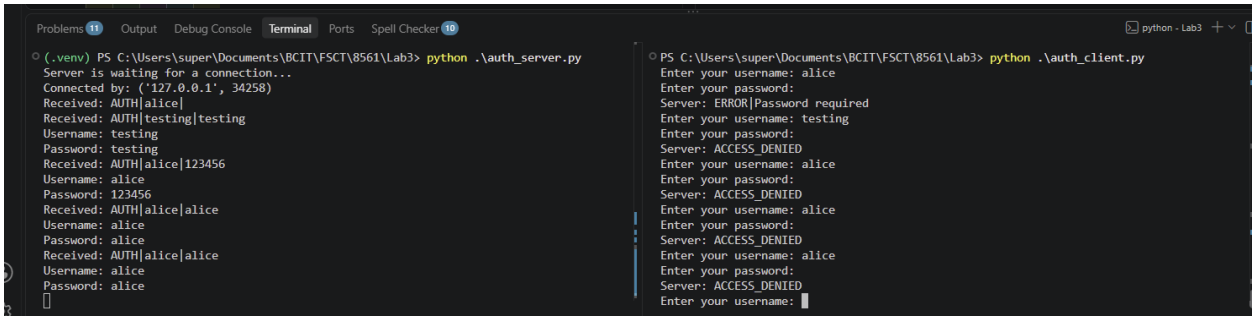


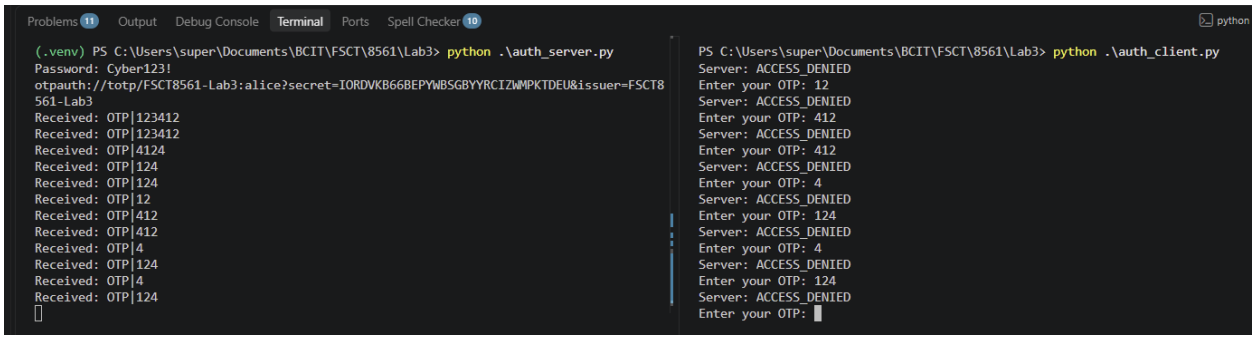
- Test 7 – Malformed Message



- Test 8 – Repeated Failed Attempts

Braeden Venne





- My application does not crash with repeated failed attempts. This is a security issue as no restrictions on failed attempts make a brute-force attack much easier to accomplish

Braeden Venne

Part 14 – Investigate Clock Synchronization

1. Why does TOTP depend on synchronized time?
    a. TOTP depends on synchronized time because both the client device and the server use the current time along with a shared secret to generate the code. If the time is different on either end, it would essentially make authentication impossible.
2. What happens when a code is submitted after its validity window?
    a. If a code is submitted after its validity window expires it is rejected by the server since it no longer matches the current expected code.
3. Why might a small amount of clock tolerance sometimes be necessary?
    a. A small clock tolerance is needed to account for slight network latency, without the small grace window, if a user enters a code right before it expires, they may be rejected due to the network delay alone.
4. What security trade-off could occur if the server accepts too many neighboring time windows?
    a. If the server accepts too many neighboring time windows, it extends the lifespan of each generated TOTP. This increases the amount of time an attacker has to intercept and use stolen TOTP or successfully guess one through brute-force.

Braeden Venne

Part 15 – Security Reflection

1. Why should passwords not be stored in plaintext?
    a. If an attacker gains access to the system or the file containing these passwords written in plaintext, they can easily get into the corresponding account, whereas if encryption was present, they would have a much more difficult time.
2. What is the purpose of hashing a password before storage?
    a. It serves as a secondary protection against bad actors who may gain access to the database or storage containing password information. If it is hashed, they will most likely not be able to do anything with it.
3. Why is plain SHA-256 alone insufficient for production password storage?
    a. SHA-256 was designed specifically for speed NOT password hashing. Its high throughput makes it vulnerable to modern hardware attacks.
4. What additional protection does TOTP provide beyond a password?
    a. TOTP provides a secondary protection to the password, it ensures that even if the attacker gets access to the password, they will not be able to get into the account as there is a rotating OTP in place.
5. How can TOTP reduce the usefulness of a captured one-time code after it expires?
    a. If they capture an expired one-time code they will simply not be able to get into the account.
6. What happens when the user and server clocks are significantly out of sync?
    a. If the client and server clocks are out of sync, the generated TOTP will not match the expected TOTP on the server, resulting in what appears as valid codes being rejected as incorrect
7. What happens if an attacker obtains the user's TOTP secret?
    a. They will be able to generate a TOTP on their end and be able to get into the users account, if they have the password already.
8. Why should repeated failed login attempts be limited or monitored?
    a. Repeated failed login attempts should be limited and monitored as it is a direct countermeasure for brute-force attacks. You can flag an IP address that is constantly spamming failed attempts and block them temporarily to investigate, and you can reduce the usefulness of brute-force tools.
9. Does `getpass()` protect the password while it travels across the network? Explain.
    a. No, this all depends on the protocols that are being used to transmit the data over to the server, if you are using HTTP for example, it will be transmitted in plaintext and an attacker can use something like Wireshark to inspect the packets and see the password. You would need to setup TLS or something equivalent if you want the password to be protected across the network.
10. What additional security would be required before deploying this authentication system in the real world?
    a. You would need TLS for packet encryption, so sensitive data is not transmitted over the network in plain text. Additionally, there should be a monitoring system set up for repeated failed login attempts and account lockout measures in place when a threshold of failed attempts is exceeded. There should also be an encrypted

database which stores hashed passwords and usernames, so user account information is protected.

Part 16 – Security Analysis

In my implementation, the server uses “hashlib.sha256()” to generate a hex digest of the password and stores a "password_hash" variable in the user dictionary. When a user authenticates using AUTH|username|password, the server runs a verification by hashing the user inputted password and compares it against the stored password hash. Adding TOTP changes the workflow into a two-step authentication sequence. The server first validates the inputted password hash, if it is rejected it returns "ACCESS_DENIED", otherwise it sends an "OTP_REQUIRED" message and waits for the user to send back an OTP|code message before issuing "ACCESS_GRANTED".

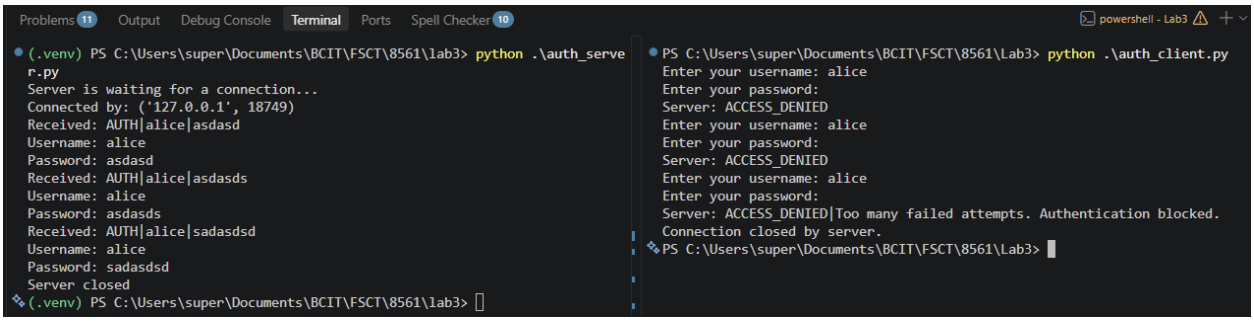
This combination helps mitigate credential stuffing and brute-force attacks. Even if an attacker was able to capture Alice’s password “Cyber123!”, they still cannot log in without the rotating 30-second token generated by pyotp. However, several attack vectors remain due to how the current system is built.

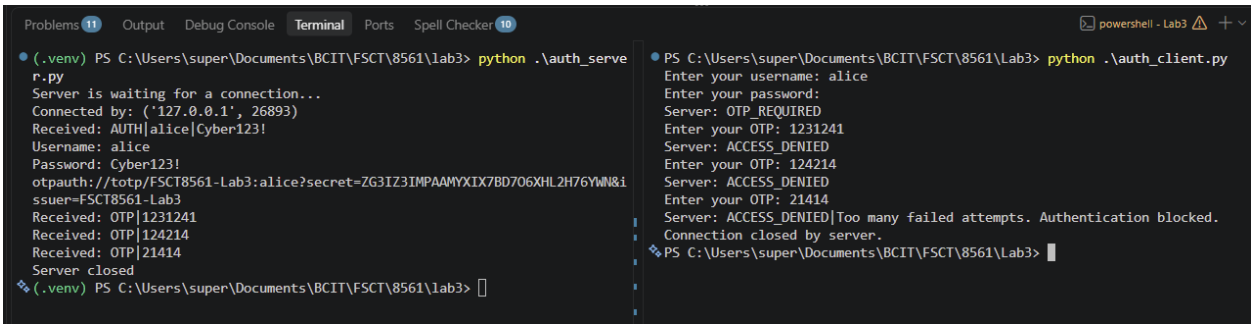
The first vulnerability in the system is that credentials and TOTP codes are transmitted over a raw, unencrypted TCP socket. Anyone running Wireshark on the network can inspect the plaintext AUTH and OTP payloads right out of the packet trace. Second, using just SHA-256 on its own has limitations, it was specifically designed for speed not hashing so its high throughput makes it vulnerable to modern hardware attacks. Third, if an attacker was able to get a hold of Alice's totp_secret, they can generate valid TOTP codes on their own machine indefinitely. Additionally, my server currently lacks rate limiting and lockouts for failed login attempts. During testing, I was able to spam incorrect password, and OTP attempts repeatedly without getting locked out, making brute-force attacks much easier to execute.

Before deploying this system in a real-world environment, several improvements are required. The raw socket should be wrapped in TLS to encrypt network traffic and prevent eavesdropping. SHA-256 should be replaced with a better suited hashing algorithm. Lastly, an account lockout policy or IP rate limiting must be implemented to block repeated failed attempts after a set threshold.

Braeden Venne

Part 17 – Challenge: Add Failed Login Protection





Challenge Questions:

1. Where should the failed-attempt counter be stored?
    a. In a real system, it should be stored on a database that ties the counter to the specific username and client IP address. If it is only tracked on the server, like it is in my code, the attacker could easily just disconnect from the server and open a new connection to bypass the limit.
2. Should password failures and OTP failures count the same way?
    a. Yes, they should both increment the counter because an attacker can use brute-force tools against either stage.
3. Why can permanent account lockout itself create a denial-of-service problem?
    a. Attackers could intentionally fail logins using a legitimate user's username, locking the real account owner out of the system.
4. How would a production system improve on this simple approach?
    a. A production system should use progressive time-based lockouts, track failure counts across both IP and username, using rate-limiting, and by notifying the user via email or SMS when suspicious login attempts occur.